

Taller 01 — Foundation Models: Comparación, Decodificación y Razonamiento

Materia: MMIA 6013 IA Generativa y Agentes **Semana:** 1 · **Sesión:** 05 (sábado, 230 min) **Peso:** 25 % de la nota final
Fecha de entrega: 19 de septiembre de 2026 **Modalidad:** individual o parejas

Contexto

En esta semana revisamos la arquitectura transformer, el ciclo de vida de un modelo fundacional y las técnicas fundamentales de prompting. El jueves se cerró con cinco palancas que actúan en el momento de la inferencia y **ninguna de las cinco toca los pesos**: cómo se elige el token, qué se pone en el prompt, qué forma tiene la salida, qué cuesta, y —la que se añadió este semestre— cuánto piensa el modelo antes de empezar a responder.

Este taller mide las cinco sobre **un solo problema tuyo**. Esa es la restricción que lo hace factible en 230 minutos: eliges **una** tarea, escribes **una** función que llama al modelo, y todo lo demás son barridos sobre esa misma función.

Antes de eso hay una **Parte 0** que no usa tu tarea y no llama a ninguna API: abre un modelo **base** de 124 M de parámetros en tu propia máquina y mira la distribución sobre la que todo lo demás opera a ciegas. Es la continuación directa del notebook del miércoles (el notebook de muestreo del miércoles), donde el muestreo se implementó sobre seis logit de juguete y sobre un modelo de bigramas.

Requisito de la tarea. El problema que elijas debe tener **respuesta verificable automáticamente** en al menos 10 casos: una cifra, una clase, un JSON con un campo que se pueda comparar. Sin eso, las Partes 2 y 4 no se pueden puntuar, porque ambas miden exactitud a lo largo de un barrido y «me pareció mejor» no es una medición. Ejemplos que sirven: extracción de campos de facturas, clasificación de tickets, problemas de aritmética o lógica con respuesta única, preguntas sobre un documento cuya respuesta está en una tabla.

Con qué modelo corre cada parte

El profesor reparte en clase una **clave de OpenAI**, y la **tabla semestral de modelos** del curso creció el 2026-09-18 con cuatro filas: una de razonamiento de OpenAI y tres que corren en tu máquina. Ahora son nueve filas y **ninguna sirve para todo**.

Parte	Con la clave de OpenAI	Sin clave, en tu máquina, a costo cero
0	— (no usa API)	<code>base_local</code> — obligatoria
1	<code>propietario_economico</code> · <code>openai_razonamiento</code>	<code>open_weight_pequeno</code> para la tercera ranura
2.a	las dos de OpenAI: una acepta el muestreo y la otra lo rechaza	las tres locales
2.b	<code>propietario_economico</code>	<code>open_weight_pequeno</code> · <code>open_weight_local</code>
3	<code>propietario_economico</code>	<code>open_weight_pequeno</code>
4	<code>openai_razonamiento</code>	<code>open_weight_razonamiento</code> (~16 GB de RAM) · <code>open_weight_pequeno</code>

Cuatro cosas que esta matriz decide por ti:

1. La Parte 1 sigue exigiendo un modelo open-weight en la tercera ranura. Tener clave no la sustituye: la comparación pierde su rango si las tres ranuras son del mismo proveedor.
2. `top_k` no se puede medir con una clave de OpenAI. La tabla lo declara `no` en las dos filas de ese proveedor, y no es una limitación de un modelo: OpenAI no expone `top_k` en ninguna de sus APIs. El barrido de `top_k` de la Parte 2.b se corre en local o no se corre.
3. Los diales locales siempre actúan, y eso les quita la mitad de la lección. En `transformers` y en Ollama los tres parámetros se aceptan y surten efecto; los estados «rechazado» y `solo_valor_por_defecto` de la Parte 2.a son propiedades de un **servicio**, no de la aritmética, y solo aparecen contra una API. Lo local **añade**; no sustituye.
4. La Parte 4 cuesta diez veces menos con la fila de razonamiento de OpenAI que con la propietaria balanceada, y su dial usa las mismas palabras. Mira el presupuesto, más abajo.

Las alternativas locales no son «la versión pobre». `open_weight_pequeno` expone un presupuesto de razonamiento en **tokens** (`thinking_budget`, 1–32768), que es la misma unidad que las APIs facturan; `low`, `medium` y `high` son etiquetas cuyo significado en tokens no está documentado por nadie. Para la gráfica de exactitud contra tokens de razonamiento de la Parte 4.a, el eje local es **más honesto** que el nominal.

Parte 0 — El modelo base y la distribución (10 %)

Objetivo

Ver con tus propios ojos la distribución sobre la que las Partes 2 y 4 solo pueden hacer inferencia estadística. El miércoles muestreaste de seis logits de juguete y de un modelo de bigramas; aquí muestreas de los logits reales de un transformer, y compruebas sobre ellos las mismas tres palancas que después irás a buscar a una API.

GPT-2 es un modelo BASE: no sigue instrucciones y aquí no se puntúa exactitud

No tiene plantilla de chat, no pasó por alineación y no sabe que existe algo llamado «tarea». No se le pide nada: se le da un **prefijo** y lo continúa. Si le escribes «Clasifica este ticket en una de tres categorías», no clasifica — sigue escribiendo el texto que en su corpus venía después de una frase así. **Eso no es un fallo tuyo, no es un fallo del modelo y no se arregla con un prompt mejor.** Tus 10 casos verificables **no entran** en esta parte: aquí se mide la distribución, no el acierto.

Preparación

`pip install torch transformers`, y la fila **base_local** de la tabla semestral: 124 M de parámetros, licencia MIT, sin *gate*, unos 500 MB, corre en CPU en segundos. Cero llamadas a una API, cero dólares, ninguna credencial.

0.a — La distribución del siguiente token (4 %)

Elige un prefijo de tu dominio y obtén los **logits** del siguiente token en un solo paso hacia adelante. Sobre esa distribución:

1. Grafica `softmax(logits / T)` para los 15 candidatos más probables, con $T \in \{0.1, 0.7, 1.0, 1.5, 2.0\}$. Es la gráfica del miércoles con logits de verdad.
2. Para cada T , reporta la **entropía** en bits y **cuántos tokens acumulan el 90 % de la masa**. Ese segundo número es `top_p = 0.9 medido`: el tamaño del núcleo no es una constante del parámetro, depende del prefijo y de la temperatura.
3. Repite los dos puntos con un prefijo donde el modelo esté muy seguro —el principio de una frase hecha— y con otro donde no lo esté. Compara los dos tamaños de núcleo.

Entregable 0.a: las gráficas, una tabla $T \times (\text{entropía, tamaño del núcleo } 0,9)$ para los dos prefijos, y 3–5 frases sobre qué le hace la temperatura a la distribución. Rige la misma prohibición que en la Parte 2.b: **sin la palabra «creatividad»**.

0.b — Las tres palancas sobre esa misma distribución (4 %)

Con `model.generate(...)` y la semilla fijada:

1. **El interruptor, primero.** Genera con `do_sample=False` y `temperature` en 0,2 y en 1,5: la salida sale idéntica. Pega las dos y explica por qué. Es el estado **«aceptado y no actúa»** de la Parte 2.a, observado en local antes de ir a buscarlo a una API que no te deja mirar.
2. **`top_k = 1 reproduce greedy`.** Con `do_sample=True`, `top_k=1`, cinco corridas: los mismos ids. Compáralo con `do_sample=False`. Aquí no se infiere de la varianza entre corridas: se comprueba contra la distribución.
3. **Dónde corta cada palanca.** Sobre la distribución de 0.a, marca en la gráfica qué tokens sobreviven a `top_k = 5` y cuáles a `top_p = 0.9`, y señala un prefijo en el que los dos conjuntos **no** coinciden. Ésa es la diferencia entre un corte de tamaño fijo y uno de masa fija, y se ve en una figura.

4. **Degeneración.** Genera 100 tokens con `do_sample=False` y pega la salida sin editar. El bucle que aparece tiene nombre y bibliografía (Holtzman et al., 2020).

Entregable 0.b: las salidas crudas de los cuatro puntos, la gráfica con los dos cortes marcados, y la comprobación de `top_k = 1` con sus cinco corridas.

0.c — El límite del modelo base (2 %)

Dale a GPT-2 **la instrucción exacta** que vas a usar en las Partes 1 a 4 y pega su salida sin editarla. En 100–150 palabras: ¿qué hace en lugar de obedecer, y qué tienen los modelos de las otras partes que a éste le falta? Esta salida es la **evidencia** de la pregunta 2 de la Parte 5, que hasta hoy se respondía de memoria.

Cuándo esta parte no vale cero. Si tu salida de 0.c es coherente y obedece, sospecha: probablemente cargaste un modelo alineado por equivocación. La fila `base_local` es un modelo base **a propósito**, y el resultado esperado es una continuación que no sirve para nada.

Fuentes de esta parte

- fila `base_local` de la **tabla semestral de modelos** y su *model card*: <https://huggingface.co/openai-community/gpt2>
- la API de generación: https://huggingface.co/docs/transformers/en/main_classes/text_generation (`do_sample`, `return_dict_in_generate`, `output_scores`)
- Raschka (2024), *Build a Large Language Model (From Scratch)*, §5.3 (p. 153): temperatura y top-k implementados a mano sobre este mismo modelo
- Alammr & Grootendorst (2024), cap. 3, «Choosing a Single Token from the Probability Distribution (Sampling/Decoding)», p. 79 (pdf 101), y cap. 6, «Temperature», p. 171 (pdf 193)
- Holtzman et al. (2020), *The Curious Case of Neural Text Degeneration*: <https://arxiv.org/abs/1904.09751>
- tu propio notebook de muestreo del miércoles

Parte 1 — Comparación de modelos (10 %)

Objetivo

Desarrollar criterio para elegir el modelo correcto para un problema dado.

Instrucciones

1. Ejecuta el mismo prompt sobre **tres modelos distintos**, elegidos por su `id` desde la **tabla semestral de modelos** del curso:
 - un modelo propietario de tamaño grande,
 - un modelo propietario o económico para producción,
 - un modelo open-weight local o servido por API.
2. Registra para cada modelo:
 - **calidad** sobre tus 10 casos verificables (exactitud medida, no impresión),
 - **latencia** en segundos, declarando qué mides (ver el material de la semana 1),
 - **costo** en USD: tokens de entrada + salida × el precio vigente **de esa fila**, con la fecha de verificación que la fila trae,
 - **observaciones cualitativas**: ¿alucina? ¿respeta el formato? ¿es verboso?
3. Produce una **tabla comparativa** y un párrafo de conclusión que justifique qué modelo elegirías en producción y por qué.

Entregables

- Código que llama a los tres modelos.
- Tabla comparativa con una **columna de fecha de verificación del precio**.
- Confirmación de que leíste los precios de la tabla semestral y no de tu memoria.

Alternativas para las tres ranuras. Con la clave del curso, las dos filas de OpenAI cubren «grande» y «económico» (`openai razonamiento` a 0,20/1,20 USD por millón y `propietario_economico` a 0,15/0,60). Si no tienes clave, las tres filas de Anthropic de la tabla siguen ahí y la tercera ranura se cubre **sin gastar nada** con `open_weight_pequeno` (<https://huggingface.co/Qwen/Qwen3-1.7B>, Apache 2.0, sin *gate*) servido por Ollama o por `transformers`. `open_weight_local` sirve igual si ya lo tienes descargado.

Dos avisos que te ahorran la tarde: **Llama y Gemma están gated en Hugging Face** —hay que aceptar la licencia con cuenta y esperar la aprobación (<https://huggingface.co/docs/hub/en/models-gated>)—, y **Qwen3.5 es multimodal**: su ejemplo oficial usa `AutoProcessor + AutoModelForMultimodalLM`, no `AutoModelForCausalLM` (<https://huggingface.co/Qwen/Qwen3.5-0.8B>). Para una tarea de texto, la generación Qwen3 de `open_weight_pequeno` es menos fricción.

Parte 2 — Decodificación: qué expone la API y qué hace el dial (20 %)

Objetivo

Separar dos cosas que se confunden todo el tiempo: **que un parámetro exista en la documentación y que el parámetro haga algo en el modelo que estás llamando.**

2.a — Verificar la matriz de exposición (8 %)

Desde el 2026-09-18, la **tabla semestral de modelos** trae en cada fila un campo `parametros_expuestos` que declara si esa fila admite `temperature`, `top_p` y `top_k`, qué dial de esfuerzo de razonamiento expone y cómo se llama su contador de tokens de razonamiento.

Ese campo sale de la documentación oficial, no de una llamada a la API. Tu trabajo es comprobarlo. Para **cada uno de los tres modelos** de la Parte 1, manda la petición y clasifica cada parámetro en uno de estos tres estados, que son tres resultados distintos:

Estado	Cómo se comprueba
Aceptado y actúa	la API responde 200 y dos valores distintos del parámetro producen distribuciones de salida distintas en <i>n</i> corridas
Aceptado y no actúa	responde 200 pero la salida no cambia entre valores extremos del parámetro
Rechazado	devuelve error; se transcribe el código y el mensaje literal

El estado del medio no es una hipótesis de laboratorio. En `transformers` hay que poner `do_sample=True` para que `temperature` y `top_p` surtan efecto: con el valor por defecto los dos parámetros se aceptan sin quejarse y no se muestrea, así que las corridas salen idénticas y es fácil concluir que «la temperatura no sirve». La tabla no puede decirte esto: solo lo dice la petición.

Entregable 2.a: una tabla de 3 modelos × 3 parámetros con **dos columnas**, lo que declara `parametros_expuestos` y lo que observaste, la fecha en que corriste la prueba, y el mensaje de error **transcrito literalmente** en cada celda rechazada. Esa fecha es parte de la respuesta, no un adorno: la matriz describe el estado de un servicio, no una propiedad de la aritmética.

Toda discrepancia entre la tabla y tu observación se reporta, y suma. La tabla puede estar vencida: es lo que declara su propio campo `verified_at`. Un hallazgo bien documentado —con la fecha, el modelo, el parámetro y el mensaje— es el mejor resultado posible de esta parte, y se corrige en la tabla del curso.

Cuatro cosas que la tabla ya te dice y que no tienes que descubrir, para que gastes las llamadas donde valen:

- Una fila tiene los tres parámetros en **solo_valor_por_defecto**: cualquier valor distinto del por defecto devuelve 400. No es un fallo de tu código.
- Dos filas tienen `top_k` en **no**, y no es una limitación de esos modelos: ese proveedor no expone `top_k` en ninguna de sus APIs. Las cuatro filas que sí lo exponen son la local y las de Anthropic con muestreo.
- Una fila entera está marcada `sin_verificar`, y es la única.** Las dos afirmaciones oficiales del proveedor se contradicen y no se concilian leyendo. Esa celda **no se puede resolver desde el escritorio**: se resuelve mandando la petición, y resolverla es el punto alto de esta parte. Si la resuelves, dilo con el mensaje literal y la fecha; el campo de la tabla se actualiza con tu resultado.
- La fila `openai_razonamiento` trae un valor de vocabulario nuevo, `no_en_esta_fila`:** el mismo proveedor expone `temperature` y `top_p` en su otra fila y los rechaza en ésta. Su procedencia es rara **a propósito** y está declarada en la nota de la fila: la lista explícita de parámetros no admitidos («`temperature`, `top_p`,

`presence_penalty`, `frequency_penalty`, `logprobs`, `top_logprobs`, `logit_bias`, `max_tokens`», para todo modelo de razonamiento salvo el mayor) está en la documentación de Azure OpenAI (<https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/reasoning>), mientras que la página de OpenAI describe `reasoning_effort` **sin decir nada del muestreo** (<https://developers.openai.com/api/docs/guides/reasoning>). Dos documentaciones del mismo modelo y solo una responde la pregunta. Con la clave del curso, comprobarlo contra `api.openai.com` es trabajo de media hora y cierra el hueco.

2.b — Barrido de temperatura y top-p (12 %)

Sobre un modelo cuya matriz de 2.a diga «**aceptado y actúa**» —la tabla semestral te dice cuáles son candidatos y tu propia 2.a lo confirma—, barre la rejilla

- `temperature` $\in \{0, 0.3, 0.7, 1.0, 1.5\} \times \text{top_p} \in \{0.5, 0.9, 1.0\}$,
- **5 corridas por celda** sobre los mismos 10 casos.

Para cada celda registra:

- **exactitud media** sobre los 10 casos,
- **estabilidad**: de las 5 corridas de un mismo caso, ¿cuántas dieron la misma respuesta?
- **longitud media** de la salida en tokens.

El barrido de `top_k` se corre en local, y no es opcional. `top_k` $\in \{1, 5, 40\}$ a temperatura fija, y comprueba que `top_k = 1` reproduce greedy: la misma salida en las 5 corridas. Ninguna fila de OpenAI expone `top_k`, así que con la clave del curso esta rejilla no se puede correr: usa `open_weight_pequeno`, `open_weight_local` (<https://github.com/ollama/ollama/blob/main/docs/api.md>, objeto `options`) o la fila `base_local` de la Parte 0. Si ya hiciste 0.b, aquí solo tienes que repetirlo sobre una tarea con respuesta verificable y decir qué cambió al pasar de una distribución a una exactitud.

Conclusión (150–250 palabras). Qué celda elegirías para tu tarea y por qué, y qué celda elegirías si la tarea fuera generar tres titulares alternativos en vez de extraer un campo. Una frase obligatoria y que se califica: explica **qué le hace la temperatura a la distribución** sin usar la palabra «creatividad».

Con $T > 0$ una salida es una muestra, no una medición. Por eso son 5 corridas por celda y por eso se reporta estabilidad. Si tus 5 corridas difieren, no falló nada.

Y $T = 0$ no es una promesa de reproducibilidad. Es la rama argmax: el mismo token en cada paso sobre los mismos logits. Que una API de producción devuelva bit a bit lo mismo depende de aritmética de punto flotante, agrupación de peticiones y versión del modelo servido, y ninguna de las tres está documentada. Si tus corridas con $T = 0$ difieren, repórtalo: es un hallazgo, no un error tuyo.

Parte 3 — Prompting estructurado (15 %)

Objetivo

Demostrar que el diseño del prompt es determinante en la calidad del output.

Instrucciones

Con el mismo modelo propietario de la Parte 1 y el mismo problema, diseña **cuatro variantes**:

1. **Zero-shot simple**: solo la instrucción.
2. **Few-shot**: 2-3 ejemplos del tipo de respuesta esperada.
3. **Chain-of-Thought**: la instrucción de razonar paso a paso antes de responder.
4. **Structured output**: el formato de salida exigido (JSON con esquema, tabla, lista).

Para cada variante: respuesta completa, exactitud sobre los 10 casos, y tokens de entrada y de salida por separado —no cuestan lo mismo—.

Conclusión (200–300 palabras) sobre cuándo usarías cada técnica.

Para la variante 4, distingue lo que el proveedor **garantiza** de lo que tú **verificaste**: pedir JSON en el prompt no garantiza nada, el modo JSON garantiza que parsea, y un esquema estricto garantiza además claves, tipos y valores del enum. Ninguno de los tres garantiza que el **valor sea el correcto**. Esa comprobación es tuya y va en el informe.

Alternativas. Con la clave del curso, `propietario_economico` da salida estructurada con esquema estricto y es la fila más barata del taller. Sin clave, las cuatro variantes corren igual en `open_weight_pequeno` — **con una diferencia que hay que reportar, no esconder**: en local, «esquema estricto» no es lo mismo. Ollama acepta un `format` con esquema JSON (<https://github.com/ollama/ollama/blob/main/docs/api.md>) y `transformers` no garantiza nada por sí solo. Si corres la variante 4 en local, di **qué mecanismo** usaste y cuántas de tus 10 salidas parsearon: es exactamente la distinción entre garantía y verificación que esta parte mide.

Parte 4 — Modelos de razonamiento y niveles de esfuerzo (25 %)

Objetivo

Medir qué compras cuando un modelo «piensa» antes de responder, y encontrar el punto donde deja de pagar.

Contexto mínimo

Un modelo de razonamiento produce, antes de su respuesta visible, una tirada de tokens internos —*reasoning tokens* en OpenAI, *thinking tokens* en Anthropic— que **ocupan ventana de contexto y se facturan como tokens de salida** aunque no se devuelvan en crudo. El nivel de esfuerzo (`reasoning.effort` / `reasoning_effort` en OpenAI, `output_config.effort` en Anthropic) es el dial que regula cuántos. No es una promesa de cuántos tokens exactos: es una señal de conducta.

Dicho de otro modo: el esfuerzo **es la quinta palanca de inferencia**, se paga en tokens de salida, y por eso es medible con el mismo cronómetro y la misma calculadora de costo de la Parte 1.

4.a — Barrido de esfuerzo (15 %)

Elige **un modelo de la tabla semestral que exponga niveles de esfuerzo** —tendrás que averiguar cuáles, exactamente igual que en la Parte 2.a— y corre tus 10 casos verificables en **al menos tres niveles** (por ejemplo `low`, `medium`, `high`; añade los que tu modelo ofrezca).

Registra por nivel:

Columna	De dónde sale
exactitud sobre los 10 casos	tu verificador
tokens de razonamiento	<code>output_tokens_details.reasoning_tokens</code> (OpenAI) · <code>usage.output_tokens_details.thinking_tokens</code> (Anthropic)
tokens de salida visibles	total de salida menos los de razonamiento
latencia	el mismo cronómetro de la Parte 1
costo USD	tokens de salida totales , razonamiento incluido, al precio de la fila

Dos gráficas, y son el entregable central del taller:

- 1. exactitud frente a tokens de razonamiento,
- 2. costo USD frente a exactitud.

Conclusión (150–250 palabras): ¿dónde está tu punto de rendimientos decrecientes? Dilo con la cifra: «a partir de *medium*, cada punto de exactitud adicional me costó X USD». Si tu curva es plana, ése también es un resultado y hay que reportarlo así.

Si tu barrido no cambia nada, comprueba antes de concluir. Un nivel de esfuerzo que se acepta y no mueve los tokens de razonamiento es el mismo caso «aceptado y no actúa» de la Parte 2.a, y se diagnostica igual: mirando el contador, no la prosa de la respuesta.

Alternativas, y son tres diales distintos.

- **Con la clave del curso:** `openai_razonamiento` expone `reasoning.effort` con `none`, `low`, `medium`, `high`, `xhigh` y `max`, y devuelve el contador en `usage.output_tokens_details.reasoning_tokens` (<https://developers.openai.com/api/docs/guides/reasoning>). Es diez veces más barata que la fila propietaria balanceada: mira el presupuesto.
- **Local, con las mismas palabras:** `open_weight_razonamiento` se conduce con `--think=low|medium|high` (<https://docs.ollama.com/capabilities/thinking>). Usa el vocabulario de OpenAI sin costar un céntimo, y pide unos 16 GB de RAM. Ojo: hay reportes abiertos de que la variante grande de esa familia dejó de obedecer el nivel (<https://github.com/ollama/ollama/issues/12589>) — es «aceptado y no actúa» en un modelo local, y se diagnostica igual.
- **Local, con un dial mejor:** `open_weight_pequeno` no expone niveles nominales sino `thinking_budget`, un presupuesto en **tokens** de 1 a 32768 (https://github.com/QwenLM/Qwen3/blob/main/docs/source/getting_started/thinking_budget.md). Corre en cualquier portátil. Si usas esta vía, tu eje horizontal de la primera gráfica deja de ser una etiqueta y pasa a ser la variable que controlas, que es **mejor** resultado: en ese caso barre al menos cinco presupuestos en vez de tres niveles.

Elijas la que elijas, la columna de **tokens de razonamiento medidos** es obligatoria. En local se cuentan los tokens del tramo `<think> ... </think>` o del campo `thinking` que devuelve `/api/chat`; el nivel declarado no es una medición.

4.b — El caso donde pensar más hace daño (10 %)

Añade a tu conjunto **tres casos fáciles contaminados**: la pregunta sigue siendo trivial, pero el enunciado lleva información irrelevante, un marco que sugiere una respuesta equivocada, o una correlación espuria plausible. Córrelos en el nivel **más bajo** y en el **más alto** de esfuerzo.

Reporta la exactitud de esos tres casos en los dos niveles y responde: ¿**empeoró**? Contrasta tu resultado —incluido el resultado negativo— con los modos de fallo que reporta Gema *et al.* (2025), «Inverse Scaling in Test-Time Compute», TMLR 12/2025 (distracción por lo irrelevante, sobreajuste al marco del problema, adopción de correlaciones espurias).

Tres casos no demuestran una ley y el informe tiene que decirlo. Lo que sí demuestra el ejercicio es el hábito: **el nivel de esfuerzo se elige por tarea y se mide, no se sube por defecto.**

Presupuesto — léelo antes de lanzar el primer barrido

Las partes con API suman alrededor de **900 llamadas**, y **no están repartidas como el trabajo**: la Parte 2.b son 750 llamadas y cuesta céntimos; la Parte 4 son 36 y es, **por llamada**, la más cara con diferencia. Eso no es una curiosidad contable, es la lección de la Parte 4: **el término que no puedes predecir —los tokens de razonamiento— es el que domina el gasto por llamada**. La Parte 0 no aparece en esta cuenta porque no hace ninguna llamada: corre entera en tu máquina.

Aritmética sobre los precios vigentes de la tabla semestral, con este supuesto declarado: 1 000 tokens de entrada y 600 de salida visible por llamada, y una salida total de 1 500 / 3 000 / 7 000 tokens en los niveles bajo / medio / alto de esfuerzo.

Parte	Dónde corre	≈ USD
Parte 0 — distribución y tres palancas	<code>base_local</code> , en tu máquina	0,00
Parte 1 — 3 modelos × 10 casos	un modelo por ranura	0,21
Parte 2.a — sondas cortas	los tres	0,04
Parte 2.b — 15 celdas × 5 corridas × 10 casos	el económico o el local	0,38
Parte 3 — 4 variantes × 10 casos	el económico	0,02
Parte 4 — 3 niveles × 10 casos + 6 contaminados	<code>openai_razonamiento</code>	0,18
Parte 4, en <code>propietario_balanceado</code>		2,10
Parte 4, en <code>propietario_grande</code>		3,51
Parte 4, en local	<code>open_weight_razonamiento</code> · <code>open_weight_pequeno</code>	0,00

Presupuesto orientativo: menos de 1 USD por grupo con la clave del curso, porque la fila de razonamiento de OpenAI cuesta 0,20/1,20 USD por millón frente a los 3/15 de la propietaria balanceada. Si trabajas con clave de Anthropic, sigue valiendo la recomendación anterior —la fila balanceada y no la grande—: el barrido de esfuerzo mide una *forma*, dónde deja de pagar el dial, y esa forma se ve igual en las dos por 1,40 USD menos.

La diferencia entre 2,10 y 0,18 no la explica el dial, la explica el precio de la fila. Las dos corren el mismo número de llamadas y salen la misma cantidad de tokens de razonamiento; lo que cambia es el USD por millón. Ésa es la Parte 1 aplicada a tu propio taller, y conviene decirlo en el informe.

Tres reglas que no se negocian:

1. **Estima antes de lanzar**, con la calculadora de la sesión 04 y los precios de la fila que vas a usar, con su fecha. La estimación va en el informe.
2. **Corre un nivel completo antes de correr los tres.** Si el contador de tokens de razonamiento del nivel alto sale mucho mayor de lo que supusiste, rehaz la cuenta antes de seguir.
3. **La cifra de arriba es aritmética sobre un supuesto, no una medición.** Los tres números de salida total son los que más pueden fallar, porque dependen del modelo y de tu tarea. Si tu medición se separa del supuesto, repórtalo: es un hallazgo de la Parte 4.a.

Parte 5 — Reflexión teórica (10 %)

Responde brevemente (100–150 palabras por pregunta):

1. Explica intuitivamente qué hace el mecanismo de atención en un transformer. ¿Por qué es importante el concepto de *query*, *key*, *value*?
2. ¿Cuál es la diferencia entre un modelo base y un modelo alineado? ¿Por qué un modelo base no es directamente útil para las aplicaciones de este curso? **Responde con tu salida de la Parte 0.c pegada delante:** esta pregunta ya no se contesta de memoria.
3. En la Parte 3, la variante con Chain-of-Thought probablemente usó más tokens que la zero-shot. ¿Cuándo vale la pena pagar ese costo y cuándo no?
4. En tu matriz de la Parte 2.a hay al menos una celda «rechazado». ¿Qué gana un proveedor al retirar un parámetro de muestreo de su API, y qué pierde quien lo usaba? Responde sin moralizar sobre el proveedor: argumenta desde lo que la palanca hacía y desde lo que la sustituye.
5. Los tokens de razonamiento de la Parte 4 se facturan y casi nunca se devuelven en crudo. Si tu modelo te mostrara la traza completa, **¿sería eso una explicación de cómo llegó a la respuesta?** Apoya tu respuesta en Lanham et al. (2023), «Measuring Faithfulness in Chain-of-Thought Reasoning» y di qué consecuencia tiene para auditar un sistema en producción.

Entregables

- **Informe en PDF** con: las gráficas de distribución de la Parte 0.a y la salida cruda de la 0.c; las tablas de las Partes 1, 2.a, 2.b y 4.a; las dos gráficas de la Parte 4.a; las respuestas de la Parte 5; y las conclusiones de cada parte.
- **Repositorio GitHub** con el código reproducible (`requirements.txt` o `environment.yml`) y el conjunto de 10 casos verificables con sus respuestas esperadas.
- **Un archivo `resultados.csv` o `.jsonl` crudo** con una fila por llamada: modelo, parámetros, corrida, tokens de entrada, tokens de salida, tokens de razonamiento, latencia, salida, acierto. Las tablas del informe se derivan de ese archivo; sin él no hay reproducibilidad que calificar. Las generaciones locales de la Parte 0 van en el mismo archivo, con la columna de costo a cero: una corrida gratis también es una corrida, y el CSV es lo que hace reproducible el barrido.

Ninguna clave de API puede aparecer en el informe, en el notebook, en una captura ni en el repositorio (política de credenciales del curso). Es un hallazgo bloqueante, no una advertencia: se leen de una variable de entorno o de un `.env` que el `.gitignore` excluye.

Criterios de evaluación

Criterio	Peso
Parte 0 — Distribución graficada y medida, las tres palancas comprobadas sobre ella y el límite del modelo base documentado	10 %
Parte 1 — Tres modelos ejecutados con calidad, latencia y costo medidos y fechados	10 %
Parte 2.a — Matriz de exposición con los tres estados, mensajes literales y fecha	8 %
Parte 2.b — Barrido de decodificación con exactitud, estabilidad y conclusión	12 %
Parte 3 — Cuatro variantes de prompting implementadas y analizadas	15 %
Parte 4.a — Barrido de esfuerzo con tokens de razonamiento contados y las dos gráficas	15 %
Parte 4.b — Casos contaminados corridos en dos niveles y contrastados con la fuente	10 %
Parte 5 — Reflexión teórica	10 %
Reproducibilidad: repositorio, resultados.csv crudo y ausencia de credenciales	10 %

Terminado no es «todo corrió». Un informe con las nueve filas llenas y sin una sola fecha de verificación, o con una tabla que no se puede reconstruir desde el CSV crudo, pierde los puntos de esas filas aunque el código sea correcto.

Anexo — la tabla semestral de modelos

Este enunciado cita las filas **por su id** —«la fila `openai_razonamiento`», «la fila `base_local`»— y quien trabaje con el PDF impreso no tiene el archivo delante. Aquí está entera.

Las dos tablas de abajo se generan desde la tabla semestral de modelos** del curso cada vez que se construye este PDF.** No están escritas a mano y no pueden desincronizarse del archivo: si un precio cambia, se regenera el PDF y cambia aquí. Compruébalo al revés también —es el hábito que la Parte 1 califica—: **el precio que uses en tu informe sale de la fila y viene con la fecha de esa fila**, no de una sola fecha para toda la tabla.

Precios, acceso y fecha de verificación

id	proveedor	modelo	entrada USD/iM	salida USD/iM	verificado	variable de entorno
propietario_g rande	anthropic	claude-opus-4-8	5,00	25,00	2026-07-10	ANTHROPIC_API_KEY
propietario_b alanceado	anthropic	claude-sonnet-4-6	3,00	15,00	2026-07-10	ANTHROPIC_API_KEY
propietario_e conomico	openai	gpt-4o-mini	0,15	0,60	2026-08-26	OPENAI_API_KEY
openai_razona miento	openai	gpt-5.6-luna	0,20	1,20	2026-09-18	OPENAI_API_KEY
juez_economico	anthropic	claude-haiku-4-5	1,00	5,00	2026-07-10	ANTHROPIC_API_KEY
open_weight_lo cal	ollama	llama3.1:8b	0 · cero real	0 · cero real	—	ninguna
base_local	huggingface	openai-community/gpt2	0 · cero real	0 · cero real	—	ninguna
open_weight_p equeno	ollama	qwen3:1.7b	0 · cero real	0 · cero real	—	ninguna
open_weight_r azonamiento	ollama	gpt-oss:20b	0 · cero real	0 · cero real	—	ninguna

Qué expone la API de cada fila

id	temperatura	top_p	top_k	dial de esfuerzo y contador
propietario_grande	solo_valor_por_defecto	solo_valor_por_defecto	solo_valor_por_defecto	output_config.effort valores: low, medium, high, xhigh, max por defecto: high contador: usage.output_tokens_details.thinking_tokens
propietario_balanceado	sin_verificar	sin_verificar	sin_verificar	output_config.effort valores: low, medium, high, max por defecto: high contador: usage.output_tokens_details.thinking_tokens
propietario_economico	si	si	no	no expone dial de esfuerzo sin contador
openai_razonamiento	no_en_esta_fila	no_en_esta_fila	no	reasoning.effort (Responses API) / reasoning_effort (Chat Completions) valores: none, low, medium, high, xhigh, max por defecto: medium contador: usage.output_tokens_details.reasoning_tokens
juez_economico	si	si	si	no expone dial de esfuerzo contador: usage.output_tokens_details.thinking_tokens
open_weight_local	si	si	si	no expone dial de esfuerzo sin contador
base_local	si	si	si	no expone dial de esfuerzo sin contador
open_weight_pequeno ✓	si	si	si	depende del motor: think (booleano) en /api/chat de Ollama; enable_thinking + thinking_budget en transformers y vLLM valores: Ollama: think true/false, transformers/vLLM: thinking_budget 1-32768 tokens por defecto: Ollama: think=false si no se pide; transformers: enable_thinking=true, thinking_budget=4000 contador: campo thinking que devuelve /api/chat en Ollama; tramo ... de la salida en transformers
open_weight_razonamiento	si	si	si	think (--think=low medium high en la CLI, campo think en /api/chat) valores: low, medium, high por defecto: medium contador: longitud en tokens del campo thinking que devuelve /api/chat

✓ **open_weight_pequeno** — comprobado contra la API el 2026-09-18, no solo contra la documentación. Qué se comprobó y qué salió, más abajo.

Cómo se lee la segunda tabla

El vocabulario de las tres columnas de muestreo tiene cuatro valores y **ninguno significa «funciona»**:

Valor	Qué declara
si	la fila admite valores distintos del que trae por defecto
no	el proveedor no lo expone en ninguna de sus APIs
no_en_esta_fila	el proveedor lo expone en otra fila suya y lo rechaza en ésta
solo_valor_por_defecto	acepta el valor por defecto; cualquier otro devuelve 400
sin_verificar	la documentación oficial no permite decidirlo: hay que mandar la petición

Advertencia de alcance, y es media Parte 2.a. Estos valores salen de la **documentación oficial de cada proveedor**, no de una llamada a la API. Describen un servicio, no una propiedad de la aritmética, y **caducan más rápido que los precios**. «El proveedor lo expone» tampoco es «el parámetro actúa»: esa segunda comprobación es empírica, es tuya, y es exactamente lo que la Parte 2.a te pide medir. Las filas marcadas con ✓ ya se comprobaron contra la API en la fecha que dice su nota; el resto, no.

Toda discrepancia que encuentres entre esta tabla y tu medición se reporta y suma.

Lo único que ya se comprobó contra una API, y qué salió

Una sola fila lleva ✓, y se comprobó el 2026-09-18 contra `qwen3:1.7b` servido por Ollama. **Ninguna fila de API está comprobada**: éstas son tuyas, y son la Parte 2.a. Lo que salió, para que sepas qué forma tiene una comprobación bien hecha:

- **Los tres diales están aceptados y actúan.** Temperatura 0 da **una** salida distinta en seis corridas y temperatura 1,6 da **seis de seis**; `top_p = 0.1` y `top_k = 1` concentran igual. Eso es una medición: dos valores extremos, varias corridas, y se compara la dispersión.
- **Un 200 no prueba ni siquiera que el parámetro exista.** Ollama acepta con `200` una clave **inventada** dentro de `options`, y tampoco valida el rango: `temperature = -5` devuelve `200`. La consecuencia para tu matriz es dura y conviene tenerla presente antes de reportar nada: un `temperature` mal escrito produce **exactamente la misma evidencia** que un parámetro real que no actúa. Si vas a declarar «aceptado y no actúa», comprueba antes cómo se escribe.
- **El dial de razonamiento de esa fila se acepta y no actúa.** `think="low"` y `think="high"` se admiten con `200` y producen **el mismo** número de tokens de razonamiento —469 en tres corridas de cada uno—, porque los niveles nominales son de otra familia de modelos. Se diagnosticó mirando **el contador**, no la prosa de la respuesta, que es exactamente lo que pide la Parte 4.a.